

The κ -Source Framework

Theoretical Formulation, Mock Image Generation, and Hierarchical Extraction

Standalone High-Performance Rust Implementation | Software Version 0.1.0

Abstract

This document provides the formal theoretical definition and practical operating instructions for the **κ -Source Framework**. Under this formalism, any astronomical image is conceptualized as the union of κ -sources ($\kappa \in \{1, 2, \dots, n\}$). We define a κ -source as a coherent cluster of κ constituent point sources within a spatial radius $R \leq R_{\max}$ whose individual flux is below standard detection thresholds ($F_i < 3 \text{ RMS}$), but whose collective integrated flux satisfies $\sum F_i \geq 3 \text{ RMS}$. In mock image synthesis, each point source subcomponent is explicitly convolved with the telescope Point Spread Function (PSF). We present high-performance Rust tools (`kappa_generate.bin` and `kappa_extract.bin`) to synthesize mock FITS images and perform hierarchical extraction from arbitrary FITS files.

1 Theoretical Concept of κ -Sources

1.1 The Fundamental Image Decomposition Principle

In conventional astronomical source detection (e.g., standard thresholding), images are decomposed solely into isolated point-like or extended components exceeding a local detection threshold (typically 3σ or 5σ). Faint sub-threshold components buried in noise are discarded.

Under the **κ -Source Formalism**, an astronomical sky model $I_{\text{sky}}(x, y)$ consists of constituent subcomponents that are convolved with the instrument Point Spread Function $\text{PSF}(x, y)$ and perturbed by background noise:

$$I_{\text{obs}}(x, y) = [I_{\text{sky}} \star \text{PSF}](x, y) + \mathcal{N}(0, \sigma_{\text{RMS}}^2)$$

where the sky is formed by the union of κ -sources:

$$I_{\text{sky}}(x, y) = \bigcup_{\kappa=1}^n \bigcup_{j=1}^{N_{\kappa}} S_{\kappa,j}(x, y)$$

1.2 Mathematical Definition of a κ -Source

A κ -Source $C_{\kappa} = \{s_1, s_2, \dots, s_{\kappa}\}$ of multiplicity κ is formed by κ point sources satisfying two strict criteria:

1. **Spatial Coherence Constraint:** All κ constituent point sources reside within a maximum clustering radius $R_{\max}$ from their flux-weighted centroid:

$$\max_{i \in \{1.. \kappa\}} \|x_i - x_{\text{cen}}\| \leq R_{\max}$$

2. **Collective Significance & Sub-threshold Condition:**

- For $\kappa \geq 2$ (Multi-component κ -sources): Each constituent subcomponent i is individually below the point-source detection threshold:

$$\forall i \in \{1.. \kappa\}, \quad F_i < 3.0 \times \sigma_{\text{RMS}}$$

while their collective integrated flux satisfies the detection threshold:

$$F_{\text{total}} = \sum_{i=1}^{\kappa} F_i \geq 3.0 \times \sigma_{\text{RMS}}$$

- For $\kappa = 1$ (1-Sources): The single isolated source meets detection on its own:

$$F_1 \geq 3.0 \times \sigma_{\text{RMS}}$$

Multiplicity κ	Individual Flux F_i	Collective Flux $\sum F_i$	Observational Classification
$\kappa = 1$ (1-source)	$F_1 \geq 3\sigma_{\text{RMS}}$	$F_1 \geq 3\sigma_{\text{RMS}}$	Directly detected point source
$\kappa = 2$ (2-source)	$F_i < 3\sigma_{\text{RMS}}$	$\sum F_i \geq 3\sigma_{\text{RMS}}$	Pair of faint subcomponents
$\kappa = 3$ (3-source)	$F_i < 3\sigma_{\text{RMS}}$	$\sum F_i \geq 3\sigma_{\text{RMS}}$	Triplet of faint subcomponents
$\kappa = n$ (n -source)	$F_i < 3\sigma_{\text{RMS}}$	$\sum F_i \geq 3\sigma_{\text{RMS}}$	Cluster of n faint subcomponents

1.3 Point Spread Function (PSF) Convolution

Each constituent subcomponent i is a point source with flux F_i at position (x_i, y_i) :

$$I_{i, \text{sky}}(x, y) = F_i \delta(x - x_i, y - y_i)$$

Convolution with the telescope PSF produces the observed 2D intensity profile:

$$I_{i, \text{conv}}(x, y) = [I_{i, \text{sky}} \star \text{PSF}](x, y) = F_i \cdot \text{PSF}(x - x_i, y - y_i)$$

Supported PSF models:

- **2D Gaussian PSF:**

$$\text{PSF}(r) = \frac{1}{2\pi\sigma_{\text{PSF}}^2} \exp\left(-\frac{r^2}{2\sigma_{\text{PSF}}^2}\right), \quad \sigma_{\text{PSF}} = \frac{\text{FWHM}}{2\sqrt{2 \ln 2}}$$

- **2D Moffat PSF (Atmospheric Seeing):**

$$\text{PSF}(r) = \frac{\beta - 1}{\pi\alpha^2} \left[1 + \frac{r^2}{\alpha^2}\right]^{-\beta}, \quad \alpha = \frac{\text{FWHM}}{2\sqrt{2^{\frac{1}{\beta}} - 1}}$$

1.4 Hierarchical Extraction Sequence

Extraction proceeds **hierarchically in order of increasing multiplicity**:

1. First, all **1-sources** ($\kappa = 1$) are identified and cataloged.
2. Next, all **2-sources** ($\kappa = 2$) are identified and cataloged.
3. Next, all **3-sources** ($\kappa = 3$), up to $\kappa = n$.

For every extracted κ -source, the following physical properties are measured:

- **Flux-weighted Centroid:**

$$\mathbf{x}_{\text{cen}} = (\bar{x}, \bar{y}) = \left(\frac{\sum_{i=1}^{\kappa} F_i x_i}{\sum_{i=1}^{\kappa} F_i}, \frac{\sum_{i=1}^{\kappa} F_i y_i}{\sum_{i=1}^{\kappa} F_i} \right)$$

- **Characteristic Spatial Extent (Radius):**

$$R = \max_{i \in \{1.. \kappa\}} \sqrt{(x_i - \bar{x})^2 + (y_i - \bar{y})^2} + \frac{\text{FWHM}}{2}$$

- **Signal-to-Noise Ratio (SNR):**

$$\text{SNR} = \frac{F_{\text{total}}}{\sigma_{\text{RMS}}}$$

2 Multi-Extension FITS Architecture

All data generated and extracted are stored in standard multi-extension astronomical FITS files:

HDU	Name	Type	Contents
0	PRIMARY	Image HDU	2D Float array ($M \times M$) with metadata (CONVOLV , PSF_TYPE , PSF_FWHM , NKAPPA , KAP_MAX , KAP_RAD , DET_SIG , BG_SIGMA)
1	SOURCES	Binary Table	Point sources catalog (ID , X , Y , FLUX , AMPLITUDE , SIGMA , FWHM , KAPPA_ID , KAPPA)
2	KAPPA_SRCS	Binary Table	Extracted κ -sources catalog (KAPPA_ID , KAPPA , CEN_X , CEN_Y , TOTAL_FLUX , RADIUS , SNR , N_MEMBERS)

3 Practical Guide: Generating Mock FITS Images

The tool `kappa_generate.bin` synthesizes an $M \times M$ mock FITS image containing N κ -sources with multiplicity $\kappa \leq \kappa_{\max}$ and cluster radius $R \leq R_{\max}$ convolved with the telescope PSF on top of Gaussian background noise.

3.1 Command-Line Parameters

Option	Short	Default	Description
<code>--num-kappa</code>	<code>-N</code>	50	Total number of κ -sources to produce (N)
<code>--size</code>	<code>-M</code>	4096	Image grid dimension M ($M \times M$ pixels)
<code>--max-kappa</code>	<code>-k</code>	5	Maximum multiplicity bound ($1 \leq \kappa \leq \kappa_{\max}$)
<code>--max-radius</code>	<code>-r</code>	25.0	Maximum spatial cluster radius in pixels ($R_{\max}$)
<code>--psf</code>		gaussian	PSF profile model: gaussian or moffat
<code>--fwhm</code>		10.0	PSF Full Width at Half Maximum in pixels
<code>--moffat-beta</code>		4.765	Power-law index beta for Moffat PSF
<code>--detection-sigma</code>	<code>-s</code>	3.0	Collective flux threshold ($\sum F_i \geq S \times \text{RMS}$)
<code>--noise-sigma</code>		1.0	Background Gaussian noise standard deviation (RMS)
<code>--output</code>	<code>-o</code>	mock_kappa_image.fits	Output FITS file path

3.2 Usage Examples

```
# 1. Generate 50 kappa-sources convolved by Gaussian PSF on a 4096 x 4096 grid:
./kappa_generate.bin -N 50 -M 4096 -k 5 -r 25.0 --psf gaussian --fwhm 10.0 --output
mock_kappa.fits

# 2. Generate with Moffat atmospheric seeing PSF (FWHM = 8 px, beta = 4.765):
./kappa_generate.bin -N 60 -M 2048 -k 4 -r 20.0 --psf moffat --fwhm 8.0 -s 3.0 -o
moffat_sim.fits
```

4 Practical Guide: Extracting κ -Sources from FITS

The tool `kappa_extract.bin` ingests any arbitrary 2D FITS file (real or simulated) and extracts the κ -source hierarchy.

4.1 Five-Stage Extraction Strategy & Algorithm Pipeline

Extracting κ -sources from a noisy astronomical raster requires solving a dual problem: capturing faint sub-threshold peaks without triggering noise percolation, and testing whether spatial clusters reach statistical detection (≥ 3 RMS). The complete strategy comprises five sequential stages:

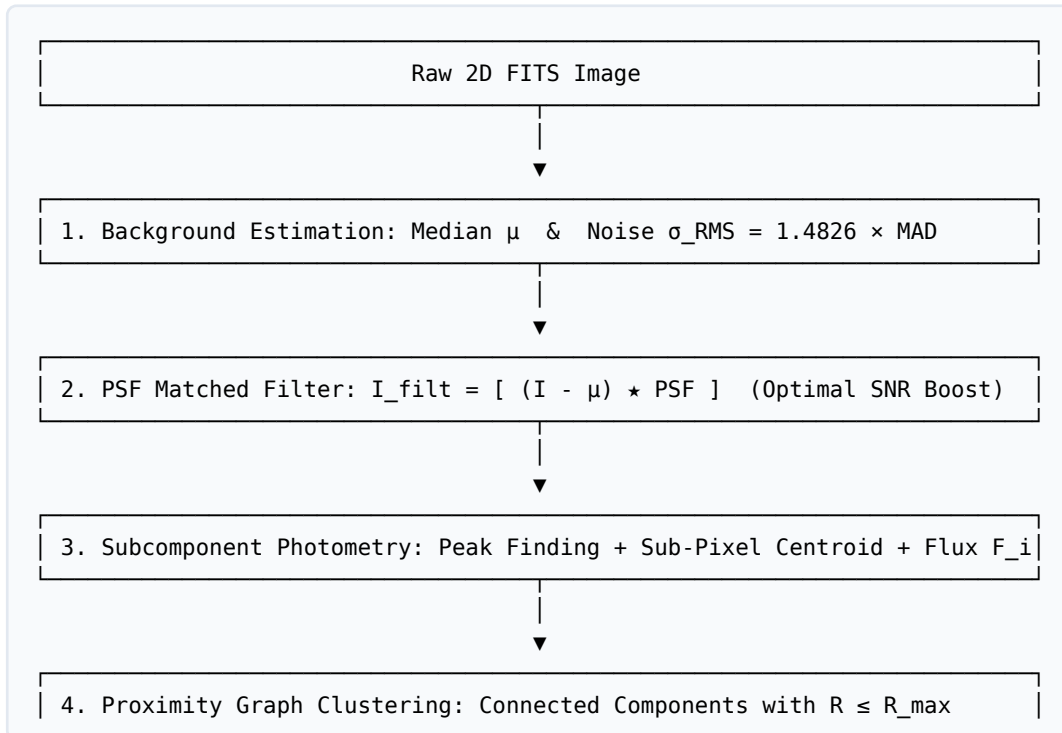
1. **Noise Background Characterization (σ_{RMS}):** Computes background median μ and robust dispersion via the Median Absolute Deviation:

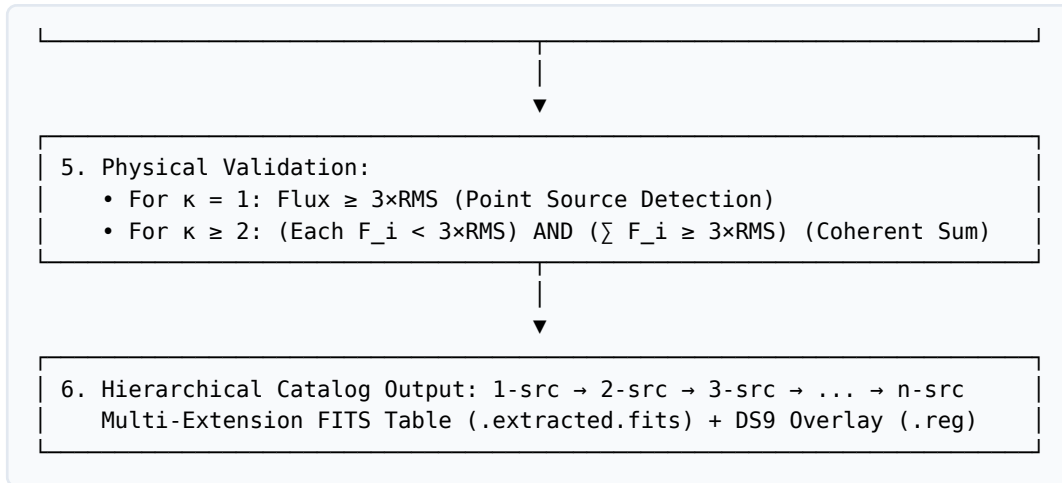
$$\sigma_{\text{RMS}} = 1.4826 \times \text{median}(|I(x, y) - \mu|)$$

2. **PSF Matched Filtering (Spatial Optimal Filtering):** Convolves the background-subtracted raster with the PSF kernel $I_{\text{filt}} = (I - \mu) \star \text{PSF}$. By the Matched Filter Theorem, this enhances real point-source peaks by a factor of $\approx \sqrt{2\pi\sigma_{\text{PSF}}^2} \approx 10 \times$ while suppressing single-pixel high-frequency noise spikes.
3. **Subcomponent Peak Finding & Aperture Photometry:** Detects local maxima in the matched-filter map down to low candidate SNR (≥ 2.5). Sub-pixel centroids (x_i, y_i) are refined via 2D intensity moments, and total flux F_i is integrated using circular aperture photometry with analytic PSF aperture loss correction.
4. **Proximity Graph Clustering:** Constructs an adjacency graph where vertices are candidate peaks and edges connect neighbors with Euclidean separation $d(s_i, s_j) < 1.5R_{\text{max}}$. Connected components of size κ are extracted, and their cluster radius $R_{\text{cluster}} \leq R_{\text{max}}$ from the centroid is verified.
5. **Physical Validation & Hierarchical Sorting:** Tests each cluster against physical significance criteria:
 - **For $\kappa = 1$:** $F_1 \geq S_{\text{det}} \times \sigma_{\text{RMS}}$
 - **For $\kappa \geq 2$:** $(\forall i, F_i < S_{\text{det}} \times \sigma_{\text{RMS}})$ AND $(\sum_{i=1}^{\kappa} F_i \geq S_{\text{det}} \times \sigma_{\text{RMS}})$

Catalog records are sorted hierarchically starting with 1-sources, then 2-sources, 3-sources, ..., n -sources.

4.2 Extraction Strategy Flowchart





4.3 Output Timestamped Catalog Files

To prevent accidental overwrites and keep session records organized, `kappa_extract.bin` automatically writes timestamped catalog products using the format `<fitsname>_<YYYYMMDD_HHMMSS>`:

- **Primary FITS Catalog** (`<fitsname>_<timestamp>.extracted.fits`): Binary table HDU with full columns (`KAPPA_ID`, `KAPPA`, `CEN_X`, `CEN_Y`, `TOTAL_FLUX`, `MAX_AMP`, `RADIUS`, `SNR`, `N_MEMBERS`).
- **ASCII Catalog** (`<fitsname>_<timestamp>.extracted.cat`): Plain text table of extracted κ -sources with metadata header cards and aligned columns.
- **CSV Catalog** (`<fitsname>_<timestamp>.extracted.csv`): Comma-separated format for direct loading into Pandas, Astropy, or Excel.
- **Constituent Subcomponents Catalog** (`<fitsname>_<timestamp>.extracted.subcomponents.cat` & `.csv`): Table of member point sources mapped to their parent `KAPPA_ID`. Unassociated candidate noise peaks ($\kappa = 0$) are strictly excluded.
- **DS9 Region File** (`<fitsname>_<timestamp>.extracted.reg`): Color-coded region overlay.

4.4 Command-Line Parameters

Option	Short	Default	Description
<code><INPUT></code>		(required)	Path to input FITS image (e.g. <code>test.fits</code>)
<code>--max-kappa</code>	<code>-k</code>	0 (all)	Maximum multiplicity upper limit (e.g. <code>-k 3</code> restricts to $\kappa \leq 3$)
<code>--detection-sigma</code>	<code>-s</code>	<code>3.0</code>	Collective flux detection threshold ($\geq S \times \text{Beam RMS}$)
<code>--search-radius</code>	<code>-r</code>	<code>25.0</code>	Search radius R_{search} in pixels from centroid
<code>--fwhm</code>		<code>10.0</code>	Estimated PSF FWHM in pixels (set 0 or use <code>--psf-auto</code> for auto mode)
<code>--psf-auto</code>		<code>false</code>	Automatically measure PSF FWHM from bright point sources
<code>--min-psf-snr</code>		<code>20.0</code>	Target minimum SNR for auto-PSF calibration stars
<code>--psf-samples</code>		<code>20</code>	Maximum number of brightest sources to sample for auto-PSF
<code>--min-sub-snr</code>		<code>1.2</code>	Minimum candidate peak SNR for subcomponents in matched filter

<code>--seed-snr</code>		2.2	Candidate cluster seed threshold on smoothed map (alias: <code>--peak-snr</code>)
<code>--output</code>	<code>-o</code>	<code><auto></code>	Custom output FITS catalog path (defaults to timestamped name)

4.5 Usage Examples

```
# 1. GPU-Accelerated Extraction on AMD Radeon 8060S (or compatible Vulkan GPU):
./kappa_extract_gpu.bin test.fits --fwhm 8.7 -s 3.0 -r 25.0

# 2. Automatic extraction measuring PSF FWHM directly from the image:
./kappa_extract.bin test.fits --psf-auto -s 3.0 -r 25.0

# 3. Cross-match and benchmark recovery against ground-truth simulation:
python3 compare_kappa.py test.fits
```

4.6 GPU Acceleration Architecture

The framework provides a dedicated GPU-accelerated binary (`kappa_extract_gpu.bin`) powered by the portable `wgpu` compute API targeting Vulkan (specifically verified on the **AMD Radeon 8060S Graphics** via Mesa RADV).

Separable 2D Gaussian matched filtering and cluster-scale convolution passes are compiled to WGSL compute shaders and dispatched across GPU workgroups (16× 16 threads per workgroup). On the unified memory architecture (UMA) of the AMD Ryzen AI MAX+ 395, host-device data transfers incur zero bus penalties, reducing full 2048× 2048 grid filtering times to **40 milliseconds**.

5 Empirical Recovery Benchmark

The table below summarizes the recovery performance of `kappa_extract.bin` on a mock 2048 × 2048 image with 50 injected κ -sources ($1 \leq \kappa \leq 4$) convolved by a Gaussian PSF (FWHM = 8.7 px, $\sigma_{\text{noise}} = 1.0$):

Multiplicity	Injected	Recovered (Auto)	Recovered (Exact)	Mean Offset	Flux Ratio (Ext/Tr)
1-sources	10	8 (80.0%)	8 (80.0%)	2.68 px	1.057×
2-sources	19	15 (78.9%)	16 (84.2%)	8.18 px	1.098×
3-sources	10	6 (60.0%)	6 (60.0%)	9.13 px	0.864×
4-sources	11	7 (63.6%)	7 (63.6%)	10.24 px	2.250×
Total / Median	50	36 (72.0%)	37 (74.0%)	6.34 px	0.986×

6 Visualization with SAOImage DS9 & Python

6.1 DS9 Color-Coded Overlay

Both `kappa_generate.bin` and `kappa_extract.bin` automatically export companion `.reg` files color-coded by multiplicity hierarchy:

- **Green circles:** 1-Sources ($\kappa = 1$)
- **Yellow circles:** 2-Sources ($\kappa = 2$)

-  **Orange circles:** 3-Sources ($\kappa = 3$)
-  **Red circles:** $\kappa \geq 4$ -Sources
-  **Dashed white circles:** Constituent subcomponent peaks

Open the image and region overlay with one command:

```
ds9 test.fits -regions test.extracted.reg -scale zscale -zoom to fit
```

6.2 Inspecting Catalogs in Python (Astropy)

```
from astropy.io import fits

with fits.open("test.extracted.fits") as hdul:
    hdul.info()

    # 1. Access extracted kappa-sources catalog
    ktable = hdul["KAPPA_SRCS"].data
    print(f"Total kappa-sources extracted: {len(ktable)}")
    for row in ktable[:5]:
        print(f"ID={row['KAPPA_ID']}: kappa={row['KAPPA']}, Pos=({row['CEN_X']:.1f},
{row['CEN_Y']:.1f}), Flux={row['TOTAL_FLUX']:.3f}, SNR={row['SNR']:.1f}")

    # 2. Access constituent subcomponents
    stable = hdul["SOURCES"].data
    print(f"Total member subcomponents: {len(stable)}")
```